

Descrierea Simplă a Proiectului

Proiectul reprezintă un software de management avansat care controlează o comunitate de 10 case inteligente (prosumatori). Fiecare locuință este dotată cu panouri fotovoltaice (PV), consumatori casnici și sisteme de stocare în baterii (BESS - Battery Energy Storage System).

Scopul principal: Aplicația acționează ca un **Broker Automat (Automated Energy Broker)**. Aceasta monitorizează bursa de energie (unde prețurile se schimbă dinamic, Piața pentru Ziua Următoare - PZU) și decide în fiecare secundă, pentru fiecare casă în parte, ce acțiune trebuie întreprinsă cu fluxurile de electricitate pentru a maximiza profitul financiar și a proteja rețeaua electrică locală împotriva suprasarcinii.

Funcționalitățile Core ale Sistemului:

- **Monitorizare IoT în timp real:** Colectarea asincronă a parametrilor electrici esențiali (producție solară, consum casnic, starea de încărcare a bateriei) de la fiecare casă din comunitate.
- **Simularea Bursei de Energie:** Generarea unui flux dinamic de prețuri bazat pe PZU, incluzând fenomene reale de piață precum vârfurile de consum sau prețurile negative (exces de producție).
- **Arbitraj Energetic Automat:** Executarea algoritmilor de decizie matematică și optimizare financiară în timp real, fără a necesita intervenție umană directă.
- **Sistem SCADA de Alertă și Protecție la Suprasarcină:** Limitarea automată a injecției de putere (curtailment) în momentele în care transformatorul local de zonă riscă să depășească limitele fizice de funcționare.
- **Dashboard Interactiv (Full-Stack):** O interfață web dinamică ce afișează grafice live cu fluxurile de putere, starea rețelei și contorul financiar al banilor economisiți sau gestionați.

2. Algoritmii Utilizați (Partea de Inginerie Electrică și Matematică)

A. Algoritmul de Bilanț Energetic al Nodului (Kirchhoff adaptat pentru Puteri)

Pentru fiecare casă i , în fiecare moment de timp t , backend-ul rulează ecuația de continuitate a puterii active:

$$P_{net,i}(t) = P_{PV,i}(t) - P_{load,i}(t) + P_{batt,i}(t)$$

Unde elementele sunt definite astfel:

- $P_{PV,i}(t)$: Puterea generată de panourile solare (kW) [valoare pozitivă].
- $P_{load,i}(t)$: Puterea absorbită de consumatorii (electrocasnicele) din casă (kW) [valoare pozitivă].
- $P_{batt,i}(t)$: Puterea bateriei (kW). Convenția de semn adoptată:
 - o $P_{batt} > 0$ (**Descărcare**): Bateria injectează putere în casă/nod.
 - o $P_{batt} < 0$ (**Încărcare**): Bateria absoarbe putere din nod, acționând ca un consumator.

- **$P_{net,i}(t)$:** Puterea netă rezultată la nivelul nodului de conectare. Dacă valoarea este pozitivă, casa are un excedent energetic și injectează în rețea. Dacă este negativă, casa trage energie din rețeaua națională.

B. Algoritmul de Arbitraj Financiar și Optimizare a Stocării

Sistemul calculează un prag de preț critic (C_{prag}) bazat pe o medie mobilă a prețurilor din ultimele 24 de ore virtuale:

$$C_{prag} = (1 / 24) * \sum_{k=1..24} Preț(t-k)$$

Logica decizională rulează automat în Java pe baza următoarelor regimuri:

- **Regim de Încărcare Forțată (Buy State):** Dacă $Preț(t) < C_{prag} * 0.75$ și Starea de Încărcare a Bateriei (SoC) $< 90\%$, bateria se încarcă la putere maximă din rețea deoarece energia este foarte ieftină.
- **Regim de Injecție Profitabilă (Sell State):** Dacă $Preț(t) > C_{prag} * 1.4$ și $SoC > 20\%$, sistemul descarcă bateria în rețea pentru a vinde energia acumulată la preț de vârf, maximizând profitul.

C. Modelul Matematic al Bateriei (State of Charge - SoC)

Pentru a asigura rigoarea tehnică, sistemul simulează chimia bateriei Li-Ion utilizând metoda numărării Coulombilor și aplicând randamente distincte pentru încărcare/descărcare ($\eta = 0.95$), respectând totodată limitele fizice stricte ($SoC_{min} = 20\%$, $SoC_{max} = 100\%$) pentru prevenirea degradării:

- **La Încărcare ($P_{batt} < 0$):**

$$SoC(t) = SoC(t-1) - [(P_{batt}(t) * \eta_{crd} * \Delta t) / E_{max}] * 100$$
- **La Descărcare ($P_{batt} > 0$):**

$$SoC(t) = SoC(t-1) - [(P_{batt}(t) * (1 / \eta_{dsc}) * \Delta t) / E_{max}] * 100$$

Unde E_{max} reprezintă capacitatea maximă a bateriei (ex: 10 kWh), iar Δt reprezintă pasul de timp al simulării.

D. Algoritmul SCADA de Protecție la Suprasarcină (Peak Shaving / Curtailment)

Acest algoritm garantează stabilitatea și siguranța infrastructurii electrice hardware. Transformatorul de distribuție local are o limită fizică strictă de putere ($P_{max_trafo} = 50 \text{ kW}$). Dacă toate cele 10 case încep să injecteze simultan energie solară sau stocată, suma puterilor poate depăși capacitatea nominală:

$$\sum_{i=1..10} P_{net,i}(t) > P_{max_trafo}$$

În acest scenariu critic, backend-ul activează instantaneu funcția de **curtailment (limitare)**: transmite comenzi de urgență către invertoare pentru a tăia producția fotovoltaică sau a forța stocarea locală, scăzând artificial injecția totală pentru a menține rețeaua sub pragul de pericol.

3. Maparea Tehnologiilor și Arhitectura Software

Tehnologie	Rol în Arhitectură	Implementare Specifică în Proiect
Java 17 & Spring Boot	Nucleul aplicației (Backend)	Găzduiește motorul de optimizare și ecuațiile matematice în clase de tip @Service. Simulează hardware-ul multi-threaded prin ScheduledExecutorService pentru a rula fire de execuție paralele asincrone pentru cele 10 case. Expune API-uri REST și endpoint-uri WebSocket.
Spring Security + JWT	Securitate și Control Acces	Protejează infrastructura critică împotriva atacurilor cibernetice. Folosește pachete securizate de tip JSON Web Token pentru autentificarea simulatoarelor/dispozitivelor. Implementează securitate la nivel de metodă (@PreAuthorize): proprietarul (ROLE_USER) vede doar casa sa, dispecerul (ROLE_ADMIN) poate opri rețeaua.
PostgreSQL & DataGrip	Persistența Datelor (Baza de Date)	DataGrip este utilizat ca IDE pentru scrierea și optimizarea schemelor SQL. PostgreSQL stochează seriile de timp (time-series data) generate la fiecare secundă. Se folosesc indecși pe coloanele de timp și ID-ul casei, alături de SQL Window Functions pentru rapoarte de consum și profitabilitate. Legătura se face prin Spring Data JPA.
Docker & Docker Compose	Infrastructură și DevOps	Containerizează aplicația în 3 medii izolate: Frontend, Backend și Bază de Date. Fișierul docker-compose.yml orchestrează pornirea corectă și ordonată a serviciilor (ex: baza de date pornește înaintea backend-ului) asigurând independența de platformă ("la mine pe laptop funcționează").
React.js / Angular	Interfața Grafică (UI / Client)	Reprezintă consola de tip SCADA Light. Se conectează la server prin WebSockets pentru a primi date în timp real fără latență. Randează grafice dinamice (Chart.js / Recharts) suprapunând curba Gauss a producției solare peste curba consumului casnic și evoluția dinamică a prețurilor.

4. Scenariu Practic pentru Demonstrație (Demo Licență)

- Ora 13:00 (Preț PZU Foarte Mic / Negativ):** Producția fotovoltaică este la maximum (6 kW), consumul e redus (1 kW). Algoritmul detectează prețul extrem de mic, blochează injecția profitabilă în rețea și comută bateria în starea de încărcare (Buy State). P_{net} devine 0, conservând energia pentru mai târziu.
- Ora 20:00 (Vârf de consum de seară - Preț Maxim):** Producția solară este zero, consumul casei crește la 4 kW. Algoritmul identifică prețul la vârf și nivelul stabil al bateriei (SoC > 20%), trecând în Sell State. Bateria descarcă 10 kW: acoperă cei 4 kW interni și livrează 6 kW în rețeaua națională la preț maxim.

3. **Activarea Protecției SCADA (Toate casele descarcă simultan):** Suma injecțiilor cumulate atinge 60 kW, depășind limita transformatorului de 50 kW. Sistemul central automat intervine în milisecunde prin WebSocket, aplicând algoritmul de curtailment: plafonează descărcarea fiecărei baterii, scăzând totalul la 43 kW și salvând transformatorul de la deteriorare.